Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Informatyki

Praca dyplomowa inżynierska

na kierunku Informatyka
w specjalności Inżynieria Wytwarzania Oprogramowania

Projekt i implementacja komponentowego silnika gry strategicznej z turowym systemem rozgrywki

Tomasz Nejman
numer albumu 331411

promotor
mgr inż. Witold Wysota

WARSZAWA 2027

Projekt i implementacja komponentowego silnika gry strategicznej z turowym systemem rozgrywki

Streszczenie. Treść streszczenia po polsku (maksymalnie jedna strona).

Słowa kluczowe: XXX, XXX, XXX

Design and implementation of a component-based strategy game engine with turn-based gameplay

Abstract. Abstract text in English (one page at most).

Keywords: XXX, XXX, XXX

Wersja w języku polskim

Politechnika Warszawska

…….....
miejscowość i data

……..
imię i nazwisko studenta

……..
numer albumu

…….……
wydział kierunek studiów

OŚWIADCZENIE

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.
Jednocześnie oświadczam, że:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.

…….....
(podpis studenta)

Wersja w języku polskim

Politechnika Warszawska

…….....
miejscowość i data

……..
imię i nazwisko studenta

……..
numer albumu

…….…….
wydział i kierunek studiów

Oświadczenie studenta w przedmiocie udzielenia licencji

Politechnice Warszawskiej

Oświadczam, że jako autor/współautor* pracy dyplomowej udzielam/nie udzielam* Politechnice Warszawskiej nieodpłatnej licencji na niewyłączne, nieograniczone w czasie, umieszczenie pracy dyplomowej w elektronicznych bazach danych oraz udostępnianie pracy dyplomowej w zamkniętym systemie bibliotecznym Politechniki Warszawskiej osobom zainteresowanym.

Licencja na udostępnienie pracy dyplomowej nie obejmuje wyrażenia zgody na wykorzystywanie pracy dyplomowej na żadnym innym polu eksploatacji, w szczególności kopiowania pracy dyplomowej w całości lub w części, utrwalania w innej formie czy zwielokrotniania.

…….....
(podpis studenta)

* niepotrzebne skreślić

Spis treści

1. Wstęp 8
1.1. Cel pracy inżynierskiej 8
1.2. Układ pracy 8
1.3. Słownik pojęć i oznaczeń 8
2. Wprowadzenie do architektury silników gier 9
2.1. Granica między silnikiem a grą 9
2.2. Od układów cyfrowych do monolitycznego kodu 9
2.3. Wzrost mocy obliczeniowej i separacja silnika od gry 9
2.4. Warstwy silnika i pętla gry 10
2.5. Zarządzanie stanem gry 10
2.6. Architektura komponentowa 10
2.7. Silniki gier strategicznych a uniwersalność kodu 10
3. Planowanie w grach turowych 12
3.1. Złożoność gier turowych 12
3.2. Minimax i przycinanie alfa-beta 12
3.3. Monte Carlo Tree Search i Statistical Forward Planning 12
3.4. Losowość i niepełna informacja 12
3.5. Paradygmat Forward Model 12
3.6. Strategie obsługi stanu: kopiowanie, cofanie, współdzielenie 12
4. Istniejące rozwiązania 13
4.1. Zakres przeglądu 13
4.2. Środowiska badawcze 13
4.2.1. Tribes 13
4.2.2. MicroRTS 13
4.2.3. Ograniczenia środowisk opartych na Javie 14
4.2.4. Stratega 14
4.3. Rozwiązania dla Heroes of Might and Magic III 14
4.3.1. VCMI 14
4.3.2. vcmi-gym i MMAI 14
4.4. Silniki komercyjne 14
4.5. Porównanie rozwiązań i wskazanie luki 14
5. Analiza wymagań 15
5.1. Odwzorowane i pominięte mechaniki gry 15
5.2. Wymagania funkcjonalne 15
5.3. Wymagania niefunkcjonalne 15
5.4. Wybór technologii 15
6. Projekt silnika 16
6.1. Architektura komponentowa silnika 16
6.2. Oddzielenie logiki gry od renderowania i obsługi wejścia 16
6.3. Maszyna stanów gry 16

6.4. Polecenia jako dane . . . 16
6.5. Interfejs Forward Model . . . 16
6.6. Projekt stanu bitwy . . . 16
6.7. Interfejs agentów . . . 16
7. Implementacja . . . 17
7.1. Środowisko i proces budowania . . . 17
7.2. Stan bitwy i jego kopiowanie . . . 17
7.3. Generowanie legalnych poleceń i przejścia między stanami . . . 17
7.4. Integracja agentów . . . 17
7.5. Warstwa prezentacji . . . 17
7.6. Testy i weryfikacja . . . 17
8. Podsumowanie . . . 18
8.1. Wnioski . . . 18
8.2. Kierunki rozwoju . . . 18
Bibliografia . . . 19
Spis rysunków . . . 20
Spis tabel . . . 21
Spis załączników . . . 22

1. Wstęp

1.1. Cel pracy inżynierskiej

1.2. Układ pracy

1.3. Słownik pojęć i oznaczeń

2. Wprowadzenie do architektury silników gier

2.1. Granica między silnikiem a grą

Zrozumienie współczesnej architektury silników gier wymaga w pierwszej kolejności zdefiniowania granicy między samym silnikiem a logiką gry. Jak zauważa Jason Gregory w monografii *Game Engine Architecture* [1], w praktyce rozróżnienie, gdzie kończy się silnik, a zaczyna gra bywa trudne, a granica pozostaje rozmyta. Do tego silniki gier rzadko powstają w próżni. Przykładowo silnik *Unreal Engine 1* (rok 1998) tworzony był specyficznie dla flagowej gry Epic Games *Unreal*.[REF NEEDED]

2.2. Od układów cyfrowych do monolitycznego kodu

Historycznie, projektanci pierwszych gier nie rozróżniali pojęć "gra" i "silnik" w ogóle, gdyż nie było możliwości myślenia na takim poziomie abstrakcji. Początkowo gry nie były w ogóle "pisane" we współczesnym znaczeniu, z wykorzystywaniem języka programowania. Analiza opisu patentowego *Video Image Positioning Control System for Amusement Device* [2] wykazuje, iż maszyny odpowiedzialne za uruchomienie gry *PONG* (rok 1972) nie posiadały procesora, pamięci operacyjnej, a tym bardziej kodu w dzisiejszym rozumieniu. Cała "logika" gry, tj. fizyka piłki, wyświetlanie punktów, była permanentnie utrwalona na płytce TTL i jej układach cyfrowych. Mechanika gry była de facto tożsama ze sprzętem komputerowym.

Sytuacja uległa zmianie wraz z upowszechnieniem się mikroprocesorów. Poziom abstrakcji wzrósł ponad okablowanie do kodu maszynowego, jednak zasoby sprzętowe były drastycznie ograniczone, co uwidacznia poniższa tabela:

Tabela 2.1. Ilość pamięci RAM wybranych urządzeń.

Urządzenie	Ilość pamięci RAM	Rok wydania
Oryginalny automat z PAC-MANem	2 KB	1980
Commodore 64	64 KB	1982
Atari 600XL	64 KB	1983

W konsekwencji tych ograniczeń programy, a w szczególności gry, stanowiły architektoniczny monolit. Przykładowo w grze *PAC-MAN* [3] logika gry, AI "duszków", rysowanie grafiki i obsługa joysticka są ze sobą całkowicie wymieszane, ze względu na wymóg intensywnej optymalizacji. Każda gra była tak naprawdę swoim własnym, unikalnym silnikiem, którego nie dało się podzielić na komponenty, czy w skuteczny sposób użyć ponownie.

2.3. Wzrost mocy obliczeniowej i separacja silnika od gry

Już w 1965 roku Gordon Moore [4] zaobserwował wykładniczy wzrost liczby tranzystorów w układach scalonych i przewidział, że utrzyma się on jeszcze przez wiele lat. Ów wzrost spowodował monumentalne i stabilne zwiększanie się mocy obliczeniowej komputerów/konsol i rozmiarów pamięci. Pozwoliło to na pojawienie się gier, wpierw pseudo-3D[REF P3D], np. *Wolfenstein 3D* (rok 1992) czy *DOOM* (rok 1993), a później true-3D[REF T3D], np. *Quake* (rok 1996 - PC), *Super Mario*

64 (rok 1996 - Nintendo 64). Oprócz tego zwiększyły się możliwości nie tylko graficzne, ale także logiczne. Gwałtowny wzrost liczby symulowanych obiektów oraz wprowadzenie przestrzeni trójwymiarowej wywołały kryzys związany z zarządzaniem globalnym stanem gry. W grach dwuwymiarowych ten stan mógł być kontrolowany przez relatywnie prostą strukturę danych. W środowisku 3D, gdzie jednocześnie musiały współistnieć systemy detekcji kolizji, kalkulacji fizyki oraz zaawansowanej sztucznej inteligencji, utrzymanie spójności danych stało się wyzwaniem. Tradycyjne, monolityczne podejście, w którym cały program miał bezpośredni dostęp do zmiennych globalnych, prowadziło do błędów synchronizacji, trudnych do wykrycia efektów ubocznych oraz uniemożliwiało skuteczne skalowanie kodu. To drastyczne skomplikowanie systemów zarządzania stanem wymusiło ostateczną separację logiki od kodu bazowego. Ponownie programiści musieli wejść na jeszcze wyższy poziom abstrakcji. Oprócz tego, jak wspominają Bishop et al. [5] (1998) tworzenie tak skomplikowanych gier od zera nie byłoby ani ekonomicznie opłacalne, ani konieczne. Definiują oni silnik gry jako zbiór elementów gry, nie mających wpływu na samą zawartość gry, tj:

- pobieranie informacji z urządzeń wejścia,
- symulowanie dynamiki obiektów,
- renderowanie obrazu,
- odtwarzanie dźwięku.

2.4. Warstwy silnika i pętla gry

2.5. Zarządzanie stanem gry

2.6. Architektura komponentowa

2.7. Silniki gier strategicznych a uniwersalność kodu

O ile powyższa taksonomia odpowiada klasycznym silnikom, szczególnie gier typu FPS, TPS, MMO, MOBA, w przypadku gier strategicznych sytuacja wygląda nieco inaczej. W przypadku gier RTS lub turowych priorytety przesuwają się z symulacji ciągłej fizyki i predykcji stanów, w stronę deterministycznego zarządzania stanem wielu obiektów, np. obsługi unikalnych wymagań strategii, takich jak zarządzanie armiami, czy specyficzne algorytmy wyszukiwania ścieżek. Ponadto, wiele gier strategicznych bazuje na planszach o dyskretnym podziale na pola, np. serie *Civilization* (sześciokątne pola), *Heroes of Might and Magic* (kwadratowe i sześciokątne pola). Ten gatunkowy rozstrzał bezpośrednio ilustruje problematykę uniwersalności kodu. Problem ten definiuje Jason Gregory w monografii *Game Engine Architecture* [1], gdzie określa silnik gry jako oprogramowanie, które jest "możliwe do rozbudowania i może być użyte jako podstawa dla wielu gier bez większych modyfikacji". Jednocześnie Gregory zaznacza, że w wielu przypadkach granica między silnikiem a grą jest rozmyta. Podkreśla on, że przesunięcie tej granicy w jedną czy drugą stronę ma bardzo jasne konsekwencje: silnik ogólnego przeznaczenia będzie bardziej uniwersalny, ale mniej przystosowany do konkretnego typu gry, zaś silnik bardziej nakierowany na dany podgatunek będzie znacznie trudniejszy lub wręcz niemożliwy do użycia w przypadku ogólnym. Ta dwojaka natura silników

powoduje, że projektowanie architektury silnika to nieustanne zarządzanie kompromisem, z jednej strony elastyczność z drugiej optymalizacja pod konkretny typ rozgrywki.

3. Planowanie w grach turowych

3.1. Złożoność gier turowych

Przede wszystkim warto wspomnieć, że w grach turowych nie sposób jest zbadać wszystkie możliwe ścieżki gry. Na przykładzie klasycznej gry w szachy Claude E. Shannon w pracy *Programming a Computer for Playing Chess* [6] wylicza liczbę możliwych rozgrywek na rząd wielkości 10^{120}, z założeniem średnio 40 ruchów per rozgrywka. Następnie z założeniem maszyny, która symuluje jedną rozgrywkę w czasie jednej mikrosekundy oblicza średni czas na wybranie pierwszego optymalnego ruchu na 10^{90} lat. Jest to liczba absolutnie przekraczająca możliwości obliczeniowe współczesnych systemów.

Nowoczesne strategiczne gry turowe przewyższają szachy w możliwej liczbie decyzji, ruchów i tur praktycznie zawsze. W złożonych grach typu 4X (eXplore, eXpand, eXploit, eXterminate), takich jak *Civilization VI*, standardowa rozgrywka wymaga rozegrania setek tur (średnio ok 350 [7]), a podczas każdej z nich gracz podejmuje wiele decyzji (średnio 10-20), każda z nich mająca wiele różnych wyników. Wskutek tego obliczenie optymalnego ruchu można uznać za niemożliwe w sensownym czasie.

3.2. Minimax i przycinanie alfa-beta

3.3. Monte Carlo Tree Search i Statistical Forward Planning

3.4. Losowość i niepełna informacja

3.5. Paradygmat Forward Model

Algorytmy sztucznej inteligencji bazują na rozgrywaniu tysięcy, jeśli nie milionów gier, symulując podejmowanie decyzji i ewaluując nowy stan. W związku z tym, dostarczany silnik musi spełniać paradygmat *Forward Model*, tj. musi dostarczać deterministyczną funkcję stan-przejście. Mając dany Stan (*S*) i Akcję (*A*) funkcja oblicza nowy stan (*S+1*). Ze względu na gigantyczną liczbę symulacji funkcja ta musi być możliwie zoptymalizowana pod względem złożoności obliczeniowej i pamięciowej. Oprócz tego wiele algorytmów drzewiastych (Minimax, MCTS) wymagają częstego duplikowania obecnego stanu gry na poddrzewa. Jednakże kopiowanie stanu gry często bywa bardzo kosztowne; przykładowo gra *StarCraft II* przez wiele lat nie dostarczała API typu *Forward Model* właśnie ze względu na złożoność obliczeniową i pamięciową[REF STARCRAFT]. Warto dodać, że optymalny *Forward Model* powinien projektować stan jako dane ciągłe w pamięci, co pozwala na klonowanie stanu za pomocą prostych operacji kopiowania bloków (cache hit) zamiast kosztownych operacji po obiektach.

3.6. Strategie obsługi stanu: kopiowanie, cofanie, współdzielenie

4. Istniejące rozwiązania

4.1. Zakres przeglądu

Mając na celu dostarczenie szybkie i wygodne FM API dla agentów AI oraz w związku z naciskiem na gatunek TBS (Turn Based Strategy) gry *Heroes of Might and Magic*, podczas rozpatrywania istniejących rozwiązań rynkowych, prac badawczych i dedykowanych silników będę skupiał się nie na skomplikowanych potokach renderowania, synchronizacji sieciowej czy algorytmach zbierania danych od użytkownika, ale głównie na tym, w jaki sposób silniki te w deterministyczny sposób zarządzają stanem gry. Co więcej, ze względu na charakter badawczy projektu mojego silnika będę również skupiał się nie tylko na możliwości prowadzenia faktycznej rozgrywki gracz-gracz, ale także na łatwości wykorzystania silnika przez agentów, na cele treningu.

4.2. Środowiska badawcze

4.2.1. Tribes

Zagadnienie dostarczenia gry z API typu FM zostało zbadane i opisane przez Diego Perez-Liebana et al. (2020) w pracy *Tribes: A New Turn-Based Strategy Games for AI Research* [8]. Jest to prosta gra turowa typu 4X, bazująca na grach typu *Civilization*, *The Battle of Polytopia*. Gra ta odbywa się na planszy 2D o rozmiarze N x N. W każdej turze gracze mogą poruszać się po planszy, atakować, rozwijać drzewo umiejętności, etc. Powodem stworzenia tej gry był właśnie narzut związany z kopiowaniem stanu w klasycznych grach 4X (i klasycznych silnikach) na rzecz trenowania agentów AI, a celem było stworzenie lekkiego mechanizmu replikowania dla algorytmów typu SFP (ang. *Statistical Forward Planning*). Oprócz tego gra dotyka aspektu częściowej możliwości obserwowania stanu gry, tj. z perspektywy danego gracza część planszy jest ukryta za tzw. "mgłą wojny". Środowisko gry *Tribes* z powodzeniem realizuje interfejs *Forward Model* udostępniając metody do kopiowania stanu gry, wykonywania ruchów. Aby zapewnić czystszy interfejs, *Tribes* wykorzystuje wzorzec *Command Pattern*, aby dokonać enkapsulacji i modularyzacji możliwych i wykonywanych ruchów. Choć nie zaimplementowano tego w *Tribes*, *Command Pattern* umożliwia również kolejkowanie ruchów lub implementację metody "undo", jeżeli ruchy są deterministyczne.

Perez-Liebana et al. (2020) skupia się głównie w *Tribes* na kombinatorycznym przyroście związanym z decyzjami globalnymi, tj. co wybudować, co zbadać, i dalekosiężnym planowaniem (SFP) w warunkach niepełnej informacji.

4.2.2. MicroRTS

Problemu wykładniczego przyrostu poddrzew w algorytmach drzewiastych dotyka również Santiago Ontañón w pracy *The Combinatorial Multi-Armed Bandit Problem and Its Application to Real-Time Strategy Games* [9]. Praca ta rozpatruje, jak sama nazwa wskazuje, problem *Combinatorial Multi-Armed Bandit*. W sytuacji gdy istnieje N jednostek i każda z nich ma M możliwych ruchów, to przestrzeń akcji w jednym kroku wynosi M^N. Ontañón skupia się na próbie oszacowania optymalnego ruchu poprzez rozważanie sposobów przeszukiwania przestrzeni i balansu eksploracja-eksploracja na podstawie autorskiego silnika gry *MicroRTS*.

Podobnie jak *Tribes*, *MicroRTS* dostarcza interfejs *Forward Model*, czyli zapewnia możliwość kopiowania stanu gry globalnym *copy* i implementuje *Command Pattern*. W przeciwieństwie do *Tribes*, *MicroRTS*, jak sama nazwa wskazuje jest grą typu RTS (Real Time Strategy), czyli czas w grze jest pseudo-ciągły (tick , a gracze podejmują decyzje w dowolnym momencie, asynchronicznie.

O ile *Tribes* optymalizuje "lekkość" samego stanu, by operacja *copy* była jak najmniej kosztowna pamięciowo i wykorzystuje płaskie struktury danych oraz prymitywy, o tyle *MicroRTS* często radzi sobie z problemem poprzez abstrakcję akcji, tj. skryptowanie zachowań jednostek, aby odciążyć *Forward Model*. W *MicroRTS* nie planuje się kroków co tick (znacznie wyższa granularność czasowa RTS), zamiast tego algorytm planuje akcje długoterminowe.

4.2.3. Ograniczenia środowisk opartych na Javie

Środowiska badawcze, takie jak opisywane wcześniej *Tribes* czy *MicroRTS*, z powodzeniem realizują interfejs *Forward Model*, ich implementacja opiera się na wysokopoziomowym, obiektowym języku programowania jakim jest Java. W Javie wszystkie obiekty klas tworzone są na stercie, a zarządzanie pamięci operate jest o garbage collector. W konsekwencji obydwa mechanizmy kopiowania stanu gry są wysoce nieoptymalne, gdyż każde głębokie kopiowanie stanu gry skutkuje kaskadą braków trafienia w pamięci podręcznej (cache miss)[REF NEEDED].

4.2.4. Stratega

4.3. Rozwiązania dla Heroes of Might and Magic III

4.3.1. VCMI

Oryginalna gra *HoMM III* była napisana głównie w języku C++ [10], ale kod do tej pory jest prywatną własnością (obecnie) firmy Ubisoft. *VCMI* to projekt open-source, mający na celu zbudowanie gry *Heroes of Might and Magic III* od zera w języku C++ [11]. Silnik *VCMI* jest budowany z myślą o replikacji gry 1 do 1, z podejściem OOP. *VCMI* nie jest jednolitym programem, dziel się na proces serwera, który autoryzuje stan gry, oraz klientów odpowiedzialnych za renderowanie i obsługę gracza. Taka separacja jest dobrym rozwiązaniem do rozgrywek sieciowych, ale dla agenta AI oznaczałby duży narzut czasowy związany z tym, że komunikacja między botem a serwerem często musi przechodzić przez warstwę sieciową. Część serwerowa *VCMI* jest ściśle ustrukturyzowana i zorientowana na dostęp do obiektów przez czyste interfejsy, jednak to wprowadza problem nieciągłości danych w pamięci. Twórcy *VCMI* nie skupiali się na tym, gdyż głównym celem było utworzenie działającej i utrzymywalnej wersji gry *Heroes III*, a nie środowiska dla agentów AI. W konsekwencji tego *VCMI* nie wprowadza interfejsu *Forward Model*.

4.3.2. vcmi-gym i MMAI

4.4. Silniki komercyjne

4.5. Porównanie rozwiązań i wskazanie luki

5. Analiza wymagań

5.1. Odwzorowane i pominięte mechaniki gry

5.2. Wymagania funkcjonalne

5.3. Wymagania niefunkcjonalne

5.4. Wybór technologii

6. Projekt silnika

6.1. Architektura komponentowa silnika

6.2. Oddzielenie logiki gry od renderowania i obsługi wejścia

6.3. Maszyna stanów gry

6.4. Polecenia jako dane

6.5. Interfejs Forward Model

6.6. Projekt stanu bitwy

6.7. Interfejs agentów

7. Implementacja

7.1. Środowisko i proces budowania

7.2. Stan bitwy i jego kopiowanie

7.3. Generowanie legalnych poleceń i przejścia między stanami

7.4. Integracja agentów

7.5. Warstwa prezentacji

7.6. Testy i weryfikacja

8. Podsumowanie

8.1. Wnioski

8.2. Kierunki rozwoju

Bibliografia

[1] J. Gregory, *Game Engine Architecture*, 3 wyd. A K Peters/CRC Press, 2018.

[2] N. K. Bushnell, „Video Image Positioning Control System for Amusement Device", pat. USA 3 793 483, 19 lutego 1974. URL: https://patents.google.com/patent/US3793483A/en

[3] Atari, Inc. „PAC-MAN Source code macro assembler", AtariWiki. URL: https://atariwiki.org/wiki/Wiki.jsp?page=PAC-MAN

[4] G. E. Moore, „Cramming more components onto integrated circuits", *Electronics*, t. 38, nr 8, s. 114–117, 19 kwietnia 1965. URL: https://www.cs.utexas.edu/~fussell/courses/cs352h/papers/moore.pdf

[5] L. Bishop, D. Eberly, T. Whitted, M. Finch i M. Shantz, „Designing a PC Game Engine", *IEEE Computer Graphics and Applications*, t. 18, nr 1, s. 46–53, 1998. DOI: 10.1109/38.637270 URL: https://www.cs.princeton.edu/courses/archive/spring2001/cs598b/papers/bishop98.pdf

[6] C. E. Shannon, „Programming a Computer for Playing Chess", *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, t. 41, nr 314, s. 256–275, marzec 1950. DOI: 10.1080/14786445008521796

[7] „Noob question, don't upvote: what's the average", Reddit, r/civ. URL: https://www.reddit.com/r/civ/comments/5h2rg6/noob_question_dont_upvote_whats_the_average/

[8] D. Perez-Liebana, Y.-J. Hsu, S. Emmanouilidis, B. Khaleque i R. Gaina, „Tribes: A New Turn-Based Strategy Game for AI Research", w: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, t. 16, 1 października 2020, s. 252–258. DOI: 10.1609/aiide.v16i1.7438 URL: https://ojs.aaai.org/index.php/AIIDE/article/view/7438

[9] S. Ontañón, „The Combinatorial Multi-Armed Bandit Problem and Its Application to Real-Time Strategy Games", w: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, t. 9, 19 października 2013, s. 58–64. DOI: 10.1609/aiide.v9i1.12681

[10] R. Gordon. „MFC to GTK", icculus.org. URL: https://www.icculus.org/~icculus/writing/linux.com/mfc_to_gtk.txt

[11] „VCMI: open source fan-made implementation of game Heroes of Might and Magic III", GitHub. URL: https://github.com/vcmi/vcmi

Spis rysunków

Spis tabel

2.1 Ilość pamięci RAM wybranych urządzeń. 9

Spis załączników

1. Instrukcja uruchomienia 23

Załącznik 1. Instrukcja uruchomienia